

TAKE-HOME ASSIGNMENT

Build a Production-Ready Payment Collection AI Agent

Role: Agent Engineer | Submission: Code + Doc + Eval

Overview

You will build a conversational AI agent that handles an end-to-end payment collection flow. The problem is intentionally underspecified in places — figuring out the right design is part of what we are evaluating.

We are specifically evaluating:

- Context management (multi-turn conversations)
- Tool calling (API integration)
- Failure handling
- Structured outputs
- System design thinking

Problem Statement

Build an AI agent that conducts the following flow with a user over chat:

1. Greet the user and prompt for their account ID
2. Look up the account via the provided API
3. Collect identity information and verify the user (see Verification Requirements)
4. Share the outstanding balance with the verified user
5. Collect card payment details
6. Process the payment via the provided API
7. Communicate the outcome clearly (success with transaction ID, or failure with reason)
8. Recap and close the conversation

What Real Users Sound Like

Users will not respond in clean, structured formats. Expect free-form, conversational input at every step — your agent must handle this gracefully.

Some examples of what real inputs look like:

Account ID

Instead of...	Expect...
ACC1001	"yeah my account number is ACC1001 I think"
ACC1001	"it's ACC 1001" or "account id: acc1001"

Full Name

Instead of...	Expect...
Nithin Jain	"my name is Nithin Jain"
Nithin Jain	"it's Nithin, Nithin Jain"
Rajarajeswari Balasubramaniam	"you can call me Raja but my full name is Rajarajeswari Balasubramaniam"

Date of Birth

Instead of...	Expect...
1990-05-14	"I was born on 14th May 1990"
1990-05-14	"DOB is May 14, 90"
1990-05-14	"14-05-1990"

Aadhaar Last 4 / Pincode

Instead of...	Expect...
4321	"last four of my Aadhaar is 4321"
400001	"pincode? it's 4 0 0 0 0 1"
9876	"Aadhaar ends with 9876, shall I give pincode instead?"

Payment Amount

Instead of...	Expect...
1000.00	"I want to pay a thousand rupees"
540.00	"just clear the full amount"
500.00	"can I do 500 for now?"

Card Details

Instead of...	Expect...
Card number: 4532015112830366	"the card number is 4532 0151 1283 0366"
Expiry: 12/2027	"expires December 2027" or "12/27"
CVV: 123	"CVV is one two three"

What This Means for Your Design

A rigid state machine that expects exact inputs will break on most of these. Think carefully about how your agent extracts intent and structured data from messy, natural language — and what role the LLM should play in that process versus deterministic code.

Verification Requirements

The agent must verify the user before proceeding to payment. Verification logic must be implemented by the agent — there is no separate verification API.

A user is verified if: Full name matches exactly **AND** at least one of the following also matches:

- Date of birth (format: YYYY-MM-DD)
- Last 4 digits of Aadhaar
- Pincode

Verification Rules

- Matching is strict — no fuzzy matching, no case-insensitive workarounds for names
- Do not proceed to any payment step until verification passes
- Handle partial inputs gracefully — guide the user to provide what is needed
- Allow reasonable retries but implement a sensible retry limit
- Do not expose account data (DOB, Aadhaar, pincode) to the user during or after verification

API Reference

Base URL: <https://se-payment-verification-api.service.external.usea2.aws.prodigaltech.com/>

POST /api/lookup-account

Fetches account details by account ID. Use the returned data to perform identity verification in-agent.

Request

```
{
  "account_id": "ACC1001"
}
```

Response — 200 OK

```
{
  "account_id": "ACC1001",
  "full_name": "Nithin Jain",
  "dob": "1990-05-14",
  "aadhaar_last4": "4321",
  "pincode": "400001",
  "balance": 1250.75
}
```

Response — 404 Not Found

```
{
  "error_code": "account_not_found",
  "message": "No account found with the provided account_id."
}
```

POST /api/process-payment

Processes a card payment against the account balance.

Request

```
{
  "account_id": "ACC1001",
  "amount": 500.00,
  "payment_method": {
    "type": "card",
    "card": {
      "cardholder_name": "Nithin Jain",
      "card_number": "4532015112830366",
      "cvv": "123",
      "expiry_month": 12,
      "expiry_year": 2027
    }
  }
}
```

Response — 200 Success

```
{
  "success": true,
  "transaction_id": "txn_1762510325322_11f14oy"
}
```

Response — 422 Failure

```
{
  "success": false,
  "error_code": "insufficient_balance"
}
```

API Error Codes

Error Code	Description
account_not_found	Provided account_id does not exist
invalid_amount	Amount is zero, negative, or has more than 2 decimal places
insufficient_balance	Amount exceeds the account's outstanding balance
invalid_card	Card number fails Luhn check, is masked, or incorrect length
invalid_cvv	CVV is wrong length (3 digits standard, 4 for Amex)
invalid_expiry	Expiry is invalid or the card has expired

Important API Notes

- cardholder_name is accepted as-is and not validated against the account holder's name
- The API validates card format, CVV, expiry, and balance — not identity
- The server does not persist balance updates. A successful payment returns a transaction ID but the account balance will remain unchanged across requests.
- Partial payments (amount < balance) are allowed.

Sample Test Accounts

Use these accounts when developing and testing your agent:

Account ID	Full Name	DOB	Aadhaar Last 4	Pincode	Balance
ACC1001	Nithin Jain	1990-05-14	4321	400001	₹1,250.75
ACC1002	Rajarajeswari Balasubramaniam	1985-11-23	9876	400002	₹540.00
ACC1003	Priya Agarwal	1992-08-10	2468	400003	₹0.00
ACC1004	Rahul Mehta	1988-02-29	1357	400004	₹3,200.50

Note on ACC1004 (Rahul Mehta)

DOB 1988-02-29 is intentionally set to a leap year date. Consider how your agent handles date validation edge cases — is this a valid date? How should your agent respond if the user provides it exactly as listed vs. a nearby but incorrect date?

Important Constraints

Hard Rules — Must Follow

- Do NOT proceed to payment without successful verification
- Do NOT expose sensitive user data (DOB, Aadhaar, pincode) unnecessarily
- Do NOT skip steps even if the user volunteers information early
- Validate all inputs before calling any API
- Verification must be strict — no fuzzy matching
- Handle incorrect or partial inputs gracefully with clear guidance

Core Requirements

1. Context Management

- Maintain full conversation state across turns
- Do not re-ask for information already provided
- Handle out-of-order information (e.g., user provides name before being asked)

2. Tool Calling

- Decide the right moment to call each API
- Construct correct, validated payloads before calling
- Handle all API responses — success, error codes, and unexpected failures

3. Verification Logic

- Implement name + secondary factor verification using account data from the lookup API

- Reject incorrect attempts clearly and count retries
- Define and enforce a retry limit — decide what happens when it is exceeded

4. Payment Handling

- Collect all required card fields: number, CVV, expiry (month + year), cardholder name
- Support partial payments — amount can be less than or equal to balance
- Interpret API error codes and communicate them clearly to the user
- Do not store or log raw card data beyond what is necessary for the API call

5. Failure Handling

- Every API failure must result in a clear, actionable user message
- Distinguish between user-fixable errors (invalid card) and terminal failures
- For retryable errors, guide the user to retry; for terminal errors, close cleanly

Agent Interface (Required)

Your agent must expose the following Python interface exactly. This is used for automated evaluation.

```
class Agent:
    def next(self, user_input: str) -> dict:
        """
        Process one turn of the conversation.

        Args:
            user_input: The user's message as a plain string.

        Returns:
            {
                "message": str # The agent's response to display to the user
            }
        """
```

Requirements

- Must maintain all conversation state internally between calls
- Each call to next() represents exactly one turn
- Must behave consistently and deterministically across repeated runs
- Must not require any external setup between turns (no manual state resets)

Sample Usage

```
agent = Agent()
agent.next("Hi")
# → { "message": "Hello! Please share your account ID to get started." }

agent.next("My account ID is ACC1001")
# → { "message": "Got it. Could you please confirm your full name?" }

agent.next("Nithin Jain")
# → { "message": "Thanks. Could you verify your date of birth or Aadhaar last 4?" }

agent.next("DOB is 1990-05-14")
# → { "message": "Identity verified. Your outstanding balance is ₹1,250.75..." }
```

Why This Interface?

We will run an LLM-based evaluator against your agent by calling `agent.next()` in a loop, simulating different user personas and flows. Your agent must handle these calls correctly without any manual intervention. Deviating from this interface will break evaluation.

Evaluation

Design a way to evaluate your system.

Include:

- Test cases:
 - Successful flow
 - Verification failure
 - Payment failure
 - Edge cases
- Metrics (e.g., success rate, correctness of tool calls)

Deliverables

1. Working Code

- `agent.py` with the Agent class implementing the interface above
- Any supporting modules (tools, validators, etc.)
- `requirements.txt` or equivalent
- `README.md` with clear setup and run instructions
- A simple CLI or script to run the agent interactively (optional but recommended)

2. Sample Conversations (in README or separate file)

- Successful end-to-end payment
- Verification failure (user exhausts retries)
- Payment failure (e.g., invalid card, expired card)
- Edge case of your choice

3. Design Document (1–2 pages)

- Architecture overview — how the agent is structured
- Key decisions made and why (e.g., LLM-driven vs. rule-based verification)
- Tradeoffs you accepted
- What you would improve with more time

4. Evaluation Approach

- Test cases covering: happy path, verification failure, payment failure, edge cases
- How you measure correctness — what does 'correct' mean for each step?
- Any automated evaluation script you built
- Observations: where does your agent struggle?

Evaluation Criteria

Your submission will be assessed across the following dimensions:

Dimension	What We Look For
System Thinking	Is there a clear, well-structured state machine? Are edge cases anticipated?
Context Handling	Does the agent track state correctly? Does it avoid re-asking questions?
Verification Logic	Is verification strict? Are retries and failure modes handled correctly?
Tool Usage	Are APIs called at the right time with correct payloads? Errors handled?
Failure Handling	Are errors communicated clearly? Does the agent recover or close gracefully?
Code Quality	Is the code readable, modular, and maintainable?
Evaluation Design	Are test cases meaningful? Is the evaluation approach thoughtful?

Submission

Submit a link to a GitHub repository (public or shared with us) containing all deliverables.

If anything in this assignment is unclear, document your assumptions in your design document. We expect some ambiguity — how you handle it is part of the evaluation.